

Paterni ponašanja u StudyBuddy sistemu

1. Uvod

StudyBuddy sistem omogućava korisnicima kreiranje sesija učenja, prijavljivanje na sesije, evidenciju prisustva, pregled statistike i upravljanje obavještenjima. Tokom razvoja sistema definisane su osnovne klase, MVC arhitektura, kao i strukturni i kreacijski paterni.

Kako sistem postaje složeniji, javlja se potreba za boljom organizacijom komunikacije između objekata i fleksibilnijim izvršavanjem različitih algoritama. Paterni ponašanja omogućavaju upravo takvu organizaciju jer definišu način na koji objekti međusobno komuniciraju i raspoređuju odgovornosti.

U StudyBuddy sistemu odabrana su dva paterna ponašanja:

- Observer
- Strategy

2. Potreba za dodavanjem paterna ponašanja

U postojećem sistemu postoje situacije u kojima promjena stanja jednog objekta treba automatski izazvati reakciju drugih objekata.

Na primjer, kada se promijeni termin sesije učenja ili kada se sesija otkaže, svi prijavljeni korisnici trebaju dobiti odgovarajuće obavještenje. Direktno povezivanje klase SesijaUcenja sa svim korisnicima dovelo bi do velikog stepena zavisnosti između klasa i otežalo održavanje sistema.

Zbog toga se uvodi Observer patern koji omogućava da objekti budu automatski obaviješteni o promjenama bez čvrstog povezivanja klasa.

Drugi problem odnosi se na generisanje različitih vrsta statistike. Sistem može prikazivati broj prisustava, ukupan broj sesija, ukupno vrijeme učenja ili druge statističke podatke. Ukoliko bi se svi algoritmi nalazili u jednoj klasi, kod bi postao nepregledan i težak za proširenje.

Zbog toga se uvodi Strategy patern koji omogućava jednostavnu zamjenu različitih algoritama za izračun statistike.

3. Moguća primjena paterna ponašanja u StudyBuddy sistemu

Observer patern

Observer patern se koristi kada promjena jednog objekta treba automatski obavijestiti više drugih objekata.

U StudyBuddy sistemu ovaj patern je pogodan za slanje obavještenja korisnicima. Kada se promijeni termin sesije, otkáže sesija ili se pojavi važna promjena, svi prijavljeni korisnici mogu biti automatski obaviješteni bez direktnog povezivanja sa klasom SesijaUcenja.

Strategy patern

Strategy patern se koristi kada postoji više različitih algoritama koji izvršavaju isti zadatak.

U StudyBuddy sistemu ovaj patern se može primijeniti na izračun statistike. Sistem može koristiti različite strategije za prikaz statističkih podataka, kao što su broj prisustava, broj organizovanih sesija ili ukupno vrijeme provedeno u učenju.

Dodavanje nove vrste statistike ne zahtijeva izmjene postojećih klasa, nego samo kreiranje nove strategije.

Command patern

Command patern se koristi za enkapsulaciju operacija u posebne objekte.

U StudyBuddy sistemu mogao bi se koristiti za operacije kao što su prijava na sesiju, otkazivanje prijave ili evidencija prisustva. Međutim, za trenutnu verziju sistema nije izabran jer bi nepotrebno povećao složenost modela.

State patern

State patern omogućava objektu da mijenja ponašanje u zavisnosti od svog stanja.

U StudyBuddy sistemu mogao bi se koristiti za upravljanje stanjima sesije (Aktivna, Završena, Otkazana). Ipak, pošto su trenutna stanja predstavljena enumeracijom, ovaj patern nije izabran za finalni dijagram.

Template Method patern

Template Method patern omogućava definisanje osnovne strukture algoritma u baznoj klasi, dok se pojedini koraci algoritma mogu prepustiti izvedenim klasama.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti za obradu različitih procesa koji imaju sličan tok izvršavanja, ali se razlikuju u pojedinim koracima. Na primjer, proces prijave na sesiju, otkazivanja prijave ili evidentiranja prisustva može imati zajedničke korake kao što su provjera korisnika, validacija podataka, izvršavanje akcije i prikaz rezultata. Različite izvedene klase mogle bi implementirati konkretne detalje pojedinih koraka.

Ipak, ovaj patern nije izabran za finalni dijagram jer je trenutna logika sistema jednostavnije predstavljena kroz postojeće metode i kontrolere.

Chain of Responsibility patern

Chain of Responsibility patern se koristi kada zahtjev prolazi kroz niz objekata, pri čemu svaki objekat može obraditi zahtjev ili ga proslijediti sljedećem objektu u lancu.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti za validaciju prijave korisnika na sesiju. Na primjer, jedan objekat bi mogao provjeravati da li je korisnik prijavljen u sistem, drugi da li sesija postoji, treći da li ima slobodnih mjesta, a četvrti da li korisnik već ima prijavu na tu sesiju.

Ovaj patern nije izabran za finalni dijagram jer bi za trenutni obim sistema uveo dodatnu složenost, dok se validacije mogu jednostavno obaviti unutar postojeće poslovne logike.

Iterator patern

Iterator patern omogućava sekvencijalni pristup elementima kolekcije bez otkrivanja njene interne strukture.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti za prolazak kroz liste sesija, korisnika, prijave ili prisustava. Na primjer, sistem može prolaziti kroz listu prijavljenih korisnika na sesiju kako bi im poslao obavještenja ili kroz listu prisustava kako bi izračunao statistiku učenja.

Ipak, ovaj patern nije izabran za finalni dijagram jer se u ASP.NET i C# okruženju za ovakve slučajeve već koriste ugrađene kolekcije i standardni mehanizmi za iteraciju.

Mediator patern

Mediator patern se koristi za centralizaciju komunikacije između više objekata, tako da objekti ne komuniciraju direktno jedni s drugima, nego preko posrednika.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti za koordinaciju između sesija učenja, korisnika, prijava i obavještenja. Na primjer, jedna posrednička klasa mogla bi upravljati procesom prijave korisnika na sesiju, provjerom slobodnih mjesta, kreiranjem prijave i slanjem obavještenja.

Međutim, ovaj patern nije izabran jer je slična uloga već djelimično pokrivena prethodno uvedenim Facade paternom, koji pojednostavljuje pristup složenijoj logici sistema.

Visitor patern

Visitor patern omogućava dodavanje novih operacija nad postojećim klasama bez mijenjanja njihove strukture.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti ako bi bilo potrebno izvršavati različite operacije nad klasama kao što su SesijaUčenja, Korisnik, PrijavaNaSesiju i Prisustvo. Na primjer, sistem bi mogao imati operacije za generisanje izvještaja, validaciju ili izvoz podataka.

Ipak, ovaj patern nije izabran jer trenutni sistem nema potrebu za velikim brojem različitih operacija nad istom strukturom objekata.

Interpreter patern

Interpreter patern se koristi za interpretaciju posebnog jezika ili pravila zapisanih u određenom obliku.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti ako bi sistem podržavao naprednu pretragu sesija pomoću posebnih izraza ili pravila. Na primjer, korisnik bi mogao unositi složene kriterije pretrage kao što su predmet, datum, tip lokacije i dostupnost mjesta, a sistem bi interpretirao taj izraz.

Ovaj patern nije izabran jer trenutni sistem ne zahtijeva poseban jezik za pretragu ili interpretaciju složenih pravila.

Memento patern

Memento patern omogućava spremanje prethodnog stanja objekta kako bi se objekat kasnije mogao vratiti u to stanje.

U StudyBuddy sistemu ovaj patern bi se mogao koristiti za vraćanje prethodnih podataka sesije nakon izmjene. Na primjer, ako moderator promijeni termin, opis ili lokaciju sesije, sistem bi mogao sačuvati prethodno stanje i omogućiti vraćanje na staru verziju.

Ipak, ovaj patern nije izabran za finalni dijagram jer trenutni sistem ne zahtijeva funkcionalnost poništavanja izmjena ili vraćanja prethodnog stanja.

4. Odabrani paterni ponašanja

Observer patern

Observer patern omogućava da objekti automatski budu obaviješteni kada se promijeni stanje drugog objekta.

U StudyBuddy sistemu Subject predstavlja klasa SesijaUcenja, dok Observer predstavlja interfejs IObserver.

Prijavljeni korisnici predstavljaju konkretne posmatrače koji implementiraju interfejs IObserver.

Klasa SesijaUcenja sadrži metode:

- attach(observer : IObserver)
- detach(observer : IObserver)
- notify()

Kada dođe do promjene termina ili otkazivanja sesije, poziva se metoda notify(), nakon čega svi prijavljeni korisnici primaju odgovarajuće obavještenje.

Prednost ovog paternu je što SesijaUcenja ne mora direktno upravljati svim korisnicima pojedinačno. Dovoljno je obavijestiti registrovane posmatrače.

Strategy patern

Strategy patern omogućava korištenje različitih algoritama kroz zajednički interfejs.

U StudyBuddy sistemu definisan je interfejs:

IStatistikaStrategy

koji sadrži metodu:

- izracunajStatistiku(korisnik : Korisnik)

Interfejs implementiraju sljedeće konkretne strategije:

- BrojSesijaStrategy
- UkupnoVrijemeUcenjaStrategy
- BrojPrisustavaStrategy

Klasa StatistikaContext koristi odabranu strategiju za izračun statistike.

Na taj način moguće je jednostavno promijeniti način izračuna bez izmjena u ostatku sistema.

Prednost ovog paternna je fleksibilnost i jednostavno proširivanje sistema novim vrstama statistike.

5. Veza između odabranih paternna

Observer i Strategy patern nadograđuju postojeću logiku sistema.

Observer patern koristi se za automatsko obavješćavanje korisnika o promjenama vezanim za sesije učenja.

Strategy patern koristi se za generisanje različitih vrsta statističkih podataka nad postojećim podacima sistema.

Na ovaj način se postiže:

- manja povezanost između klasa,
- lakše proširivanje sistema,
- bolja organizacija poslovne logike,

- veća fleksibilnost pri dodavanju novih funkcionalnosti.

6. Zaključak

Dodavanjem Observer i Strategy paterna StudyBuddy sistem postaje fleksibilniji, pregledniji i lakši za održavanje.

Observer patern omogućava automatsko obavještanje korisnika o promjenama sesija učenja, dok Strategy patern omogućava jednostavnu zamjenu različitih algoritama za izračun statistike.

Ovi paterni ne mijenjaju postojeći model sistema, nego ga proširuju dodatnim klasama koje unapređuju komunikaciju između objekata i organizaciju poslovne logike.